

IT Olympics C# Questionnaire

EASY:

1. Strange Pair

A pair of strings form a strange pair if both of the following are true:

The 1st string's first letter = 2nd string's last letter.

The 1st string's last letter = 2nd string's first letter.

Create a function that returns true if a pair of strings constitutes a strange pair, and false otherwise.

Examples

`IsStrangePair("ratio", "orator")` → “ratio” is a Strange Pair with “orator”

`IsStrangePair("sparkling", "groups")` → “sparkling” is a Strange Pair with “groups”

`IsStrangePair("bush", "hubris")` → “bush” is not a Strange Pair with “hubris”

`IsStrangePair("", "")` → “” is a Strange Pair with “”

Notes:

It should work on a pair of empty strings (they trivially share nothing).

2. Sum of the Two Smallest Numbers

Create a function that takes an array of numbers and returns the sum of the two lowest positive numbers.

Examples:

`SumSmallest([19, 5, 42, 2, 77])` → Sum: 7

`SumSmallest([10, 343445353, 3453445, 3453545353453])` → Sum: 3453455

`SumSmallest([2, 9, 6, -1])` → Sum: 8

`SumSmallest([879, 953, 694, -847, 342, 221, -91, -723, 791, -587])` → Sum: 563

`SumSmallest([3683, 2902, 3951, -475, 1617, -2385])` → Sum: 4519

Notes:

- Don't count negative numbers.
- Floats and empty arrays will not be used in any of the test cases.

3. Remove Duplicates from an Array

Create a function that takes an array of items, removes all duplicate items and returns a new array in the same sequential order as the old array (minus duplicates).

Examples:

`RemoveDuplicate([1, 0, 1, 0]) → [1, 0]`

`RemoveDuplicate(["The", "big", "cat"]) → ["The", "big", "cat"]`

`RemoveDuplicate(["John", "Taylor", "John"]) → ["John", "Taylor"]`

Notes:

- Tests contain arrays with both strings and numbers.
- Tests are case sensitive.
- Each array item is unique.

AVERAGE:

1. Pirate's Tale

Captain Hook and his crew are currently resting at Origin Shore. They are about to embark on their next adventure to an undisclosed location (x, y) to find treasure.

Captain Hook's ship can only move exactly north, south, east or west. It takes exactly 1 day for the ship to travel 1 unit in one of the four cardinal directions.

After every 5 days, the crew will take one day of rest.

Given the location of the treasure, find out how long it takes for Captain Hook and his crew to find the treasure. The ship is currently at coordinate (0, 0).

NumberOfDays([3, 5]) => 9 days

// Since: 3 days east + 2 days north (5 days passed) + 1 day of rest + 3 days north

NumberOfDays([-4, -1]) => 5 days

// Since 4 days west + 1 day south

Examples:

NumberOfDays([10, 10]) →

Number of days required to travel: 23 day/s

Number of Rest days: 3 day/s

NumberOfDays([3, 3]) →

Number of days required to travel: 7 day/s

Number of Rest days: 1 day/s

NumberOfDays([-10, -9]) →

Number of days required to travel: 22 day/s

Number of Rest days: 3 day/s

NumberOfDays([-1, -2]) →

Number of days required to travel: 3 day/s

Number of Rest days: 0 day/s

Notes:

- Print the total number of Travel days and Rest days

2. The Prime Number Generator

You are tasked with creating a program that generates prime numbers within a specific range. Your program should take an input integer N and find all prime numbers between 1 and N.

Your task is to implement the `GeneratePrimes` function to achieve this. The function should return a list of prime numbers between 1 and N (inclusive).

Please complete the following:

Implement the `GeneratePrimes` function that takes an integer N as input and returns a list of prime numbers within the range of 1 to N.

Create a program to take user input for N, call the **`GeneratePrimes`** function, and display the list of prime numbers as the output.

Sample Input:

Enter the value of N: 20

Sample Output:

Prime numbers between 1 and 20: 2, 3, 5, 7, 11, 13, 17, 19

3. The Lexicographic Sequence Generator **(To be finalized)**

Robin is working on a project that requires generating lexicographically sorted sequences from a set of characters. He needs to generate these sequences, considering mixed capitalization and ignoring spaces. Can you write a program to help him with this task?

Please complete the following:

Implement a function **`GenerateLexicographicSequences`** that takes a set of characters as input and returns a list of unique lexicographically sorted sequences using the provided characters.

Create a program that takes user input for a set of characters, calls the **`GenerateLexicographicSequences`** function, and displays the list of unique sequences found.

Sample Input:

Enter the set of characters: abc ABC 123

Sample Output:

Unique lexicographically sorted sequences found: 123ABCabc, 123ABCacb, 123ABCbac, 123ABCbca, 123ABCCab, 123ABCcba, 123ABaCbc, 123ABaCcb, 123ABabCc, 123ABabcC

Notes:

- Provide at least 10 to 20 sequences.

Explanation:

For the given input set of characters, the program generates unique lexicographically sorted sequences while considering mixed capitalization and ignoring spaces. The sequences are presented in lexicographic order, providing Robin with the sorted sequences he needs for his project.

This question challenges you to design an algorithm to generate and sort sequences, considering mixed capitalization and ignoring spaces.

DIFFICULT (HARD)

1. Rolling Cipher

In the world of cryptographers, two talented individuals, Joshua and James, were always fascinated by ciphers and secret messages. They decided to create their very own encryption method, and they called it the **"Rolling Cipher."**

The Rolling Cipher is a unique encryption technique that involves rolling each character in a message forward or backward a specified number of times. In this world of secret messages, you can think of the letters as part of a continuous loop, so rolling a character backward once will yield 'z,' and rolling 'z' forward once will give you 'a.' The rolling continues until the specified number of times is reached.

Creating the Rolling Cipher:

Joshua and James decided to use only lowercase letters, and there were no spaces in their messages. Each character in the message is rotated the same number of times. Rolling 26 times in either direction should bring you back to where you started.\

Your Task:

Now, they want you to help them create a function that can perform this Rolling Cipher. They need your assistance to ensure their secret messages remain secure. Your task is to implement a function that accepts a string and a number 'n' and returns the message after rolling each character.

Function Signature:

string RollingCipher(string message, int n)

Examples:

- RollingCipher("abcd", 1) → "bcde"
(The letters are rolled forward by one position, creating "bcde")
- RollingCipher("abcd", -1) → "zabc"
(The letters are rolled backward by one position, creating "zabc")
- RollingCipher("abcd", 3) → "defg"
(The letters are rolled forward by three positions, creating "defg")
- RollingCipher("abcd", 26) → "abcd"
(Rolling 26 times in either direction brings you back to the original message)

Notes:

- All letters are in lowercase.
- There are no spaces in the message.
- Each character is rotated the same number of times.

2. Fibonacci Word

A Fibonacci Word is a specific sequence of binary digits (or symbols from any two-letter alphabet). The Fibonacci Word is formed by repeated concatenation in the same way that the Fibonacci numbers are formed by repeated addition.

Create a function that takes a number n as an argument and returns the first n elements of the Fibonacci Word sequence.

If $n < 2$, the function must return "invalid".

Examples:

`FiboWord(1)` → "invalid"

`FiboWord(3)` → "b, a, ab"

`FiboWord(7)` → "b, a, ab, aba, abaab, abaababa, abaababaabaab"